Python Introduction and Data Types - Complete Guide

From Beginner Basics to Advanced Concepts

PART 1: INTRODUCTION TO PYTHON

1. Python Fundamentals

1.1 What is Python?

1. What is Python? Define it in technical terms.
2. Why was Python created? What was the initial purpose?
3. Who created Python and when?
4. What are the main design philosophies of Python?
5. What does "batteries included" mean in the context of Python?
6. Is Python a compiled or interpreted language? Explain the differences.
7. What is the difference between CPython, PyPy, Jython, and IronPython?
8. Why is Python slower than languages like C or Java?
9. What are the advantages of Python?
10. What are the disadvantages of Python?

1.2 Python Versions and Evolution

11. What are the major differences between Python 2 and Python 3?
12. Why was Python 2 deprecated?
13. When did Python 2 reach end-of-life?
14. What are some significant features introduced in Python 3.6, 3.8, 3.9, 3.10, and 3.11?
15. What is the current stable version of Python?
16. How often does Python release new versions?
17. What is the Python Enhancement Proposal (PEP)?
18. What is PEP 8 and why is it important?
19. What is the difference between `python` and `python3` commands?

1.3 Installation and Environment Setup

20. How do you install Python on Windows, macOS, and Linux?
21. What is a virtual environment and why is it important?
22. How do you create a virtual environment using `venv`?

23. What is the difference between `venv` and `virtualenv`?
24. What is `pip` and how do you use it?
25. How do you check the Python version from the command line?
26. What is the Python Path (PYTHONPATH)?
27. How do you add a directory to the Python path?
28. What is an IDE? Name some popular Python IDEs.
29. What is the difference between an IDE and a text editor?

1.4 Running Python Code

30. What is the Python interpreter and how does it work?
31. How do you run Python code from the command line?
32. What is the Python interactive shell (REPL)?
33. How do you run a Python script from a `.py` file?
34. What happens when you import a module? Does it execute the code?
35. What is the `__main__` variable used for?
36. What is the difference between `python script.py` and `python -m script`?
37. How do you get a list of all built-in functions in Python?
38. What is the `dir()` function used for?
39. What is the `help()` function used for?

1.5 Python Syntax and Structure

40. What are the key syntax rules of Python?
41. Why is indentation important in Python?
42. What is the difference between tabs and spaces for indentation?
43. What is a code block? How is it defined in Python?
44. What is a statement vs. an expression?
45. What are keywords in Python? Give 10 examples.
46. What are identifiers? What are the rules for naming identifiers?
47. Can you use Python keywords as variable names?
48. Are Python variable names case-sensitive?
49. What are comments in Python and how do you write them?
50. What is the difference between single-line and multi-line comments?
51. Can you use `"""` as a comment? What does it actually do?
52. What is a docstring? How is it different from a comment?
53. What is PEP 257 about docstrings?

1.6 Variables and Memory

54. What is a variable in Python?
55. How do you declare a variable in Python?
56. Are variable types declared before assignment in Python? Why?
57. What is type inference in Python?
58. Can you change the type of a variable after assignment?
59. What is the difference between a variable and a value?
60. How does Python manage memory for variables?
61. What is the difference between mutable and immutable objects?
62. How do you check the type of a variable using `type()`?
63. What is the difference between `type()` and `isinstance()`?

1.7 Input and Output

64. How do you take input from a user in Python?
65. What does `input()` return? Is it a string or can it be other types?
66. How do you convert user input to an integer or float?
67. What is the basic syntax of the `print()` function?
68. What are some common parameters of the `print()` function?
69. What is the default separator in `print()`?
70. What is the default end character in `print()`?
71. How do you print without a newline at the end?
72. How do you print multiple variables in one line?
73. What is string formatting in Python? Name the methods.
74. What is the difference between `%` formatting, `.format()`, and f-strings?
75. How do you use f-strings (formatted string literals)?
76. What is a newline character in Python?
77. What is the escape character in Python?

1.8 Basic Error Handling

78. What is an exception in Python?
79. What is the difference between an error and an exception?
80. What are syntax errors vs. runtime errors?
81. What is the `try-except` block used for?
82. What is the basic syntax of exception handling?

83. Can you have multiple `except` blocks? Why would you?
84. What is the `else` clause in exception handling?
85. What is the `finally` clause used for?
86. What is the difference between catching a specific exception and catching all exceptions?
87. What is the `raise` statement used for?
88. How do you create custom exceptions?

PART 2: PYTHON DATA TYPES

2. Overview of Python Data Types

2.1 Introduction to Data Types

89. What is a data type?
90. Why are data types important in programming?
91. How many built-in data types does Python have?
92. What are the major categories of Python data types?
93. What is the difference between primitive and composite data types?
94. What is the difference between immutable and mutable data types?
95. How do you check the type of a value using `type()`?
96. How do you convert between data types (type casting)?
97. What is implicit type conversion?
98. What is explicit type conversion?

---

3. NUMERIC DATA TYPES

3.1 Integers

99. What is an integer?
100. How do you define an integer in Python?
101. What is the range of integers in Python?
102. Can integers be negative in Python?
103. How do you represent very large integers in Python?
104. What is the difference between regular integers and long integers?
105. Can you perform arithmetic operations on integers? Give examples.
106. What happens when you divide two integers using `/`?
107. What happens when you divide two integers using `//`?

108. How do you check if a number is an integer using `isinstance()`?
109. What is the `int()` function used for?
110. What is the output of `int("123")` and `int("123.45")`?
111. Can you convert a float to an integer? What happens to decimal places?
112. What is the output of `int(3.9)` and `int(-3.9)`?

3.2 Floats

113. What is a floating-point number?
114. How do you define a float in Python?
115. What is the difference between `int` and `float`?
116. What is IEEE 754 floating-point representation?
117. What is floating-point precision and why is it important?
118. What is the output of `0.1 + 0.2` in Python?
119. Why does `0.1 + 0.2 == 0.3` return `False`?
120. How do you compare floating-point numbers safely?
121. What is the `math.isclose()` function?
122. What is scientific notation? Give an example.
123. How do you represent `1.5e10` in Python?
124. What is the `float()` function used for?
125. Can you convert a string to a float using `float()`?
126. What is the output of `float("3.14")` and `float("inf")`?
127. What are special float values: `inf`, `-inf`, and `nan`?
128. How do you check if a float is `inf` or `nan`?

3.3 Complex Numbers

129. What are complex numbers in Python?
130. How do you define a complex number?
131. What is the syntax for complex numbers: `a + bj`?
132. What are the `real` and `imag` attributes of complex numbers?
133. Can you perform arithmetic operations on complex numbers?
134. What is the output of `(3+4j) + (1+2j)`?
135. What is the output of `(3+4j) * (1+2j)`?
136. Can you use comparison operators on complex numbers?
137. What is the `complex()` function used for?

138. What is the output of `complex(3, 4)` and `complex("3+4j")`?

3.4 Numeric Operations and Functions

139. What is the `abs()` function? Give examples for int, float, and complex.
140. What is the `round()` function? How does it work?
141. What is the output of `round(2.5)` and `round(3.5)` and why?
142. What is the `pow()` function and how is it different from `**`?
143. What is the `min()` and `max()` function?
144. What is the `sum()` function?
145. How do you generate random numbers in Python?
146. What is the `random` module?
147. What is the difference between `random.random()`, `random.randint()`, and `random.choice()`?

3.5 Numeric Type Conversions

148. How do you convert an integer to a float?
149. How do you convert a float to an integer?
150. How do you convert a string to an integer?
151. How do you convert a string to a float?
152. What happens when you convert a float with decimals to an integer?
153. What is the output of `int(3.7)`, `int(-3.7)`, `int("10")`, `int("10.5")`?
154. Can you convert a complex number to int or float?
155. What is the output of `bool(0)`, `bool(1)`, `bool(0.0)`, `bool(0.1)`?

---

4. BOOLEAN DATA TYPE

4.1 Basics of Boolean

156. What is a Boolean?
157. What are the two Boolean values in Python?
158. How do you create a Boolean value?
159. What is the `bool()` function?
160. What values are considered `True` and what values are considered `False`?
161. What is the output of `bool(0)`, `bool(1)`, `bool("")`, `bool("Hello")`?
162. What is the output of `bool([])`, `bool([1])`, `bool(None)`?

163. Can you perform arithmetic operations on Booleans?
164. What is the output of `True + True` and `True * 5`?
165. What is the difference between `True == 1` and `True is 1`?

4.2 Boolean Logic and Operations

166. How do you use Boolean values in conditional statements?
167. What are logical operators and how do they work with Booleans?
168. What is the output of `True and False`, `True or False`, `not True`?
169. What is De Morgan's Law and how does it apply to Boolean operations?
170. How do you combine multiple conditions?

4.3 Truthy and Falsy Values

171. What is a truthy value?
172. What is a falsy value?
173. List all falsy values in Python.
174. List all truthy values in Python.
175. What is the output of `if 1:`, `if 0:`, `if "":`, `if "Hello":`?
176. What is the output of `if []:`, `if [1]:`, `if None:`?
177. How can you use truthy/falsy values for default values?
178. What is the output of `5 or 10`, `0 or 10`, `"" or "Default"`?

---

5. STRING DATA TYPE

5.1 String Basics

179. What is a string in Python?
180. How do you create a string?
181. What is the difference between single quotes, double quotes, and triple quotes?
182. What are triple-quoted strings used for?
183. Can you use both single and double quotes in a string? How?
184. What is the `str()` function?
185. What is the output of `str(123)`, `str(12.34)`, `str(True)`?
186. What is string concatenation?
187. What is the `+` operator used for with strings?
188. What is the `*` operator used for with strings?
189. What is the output of `"Hello" + " " + "World"` and `"Hi" * 3`?

5.2 String Escape Sequences

190. What is an escape sequence?
191. What are common escape sequences in Python?
192. What is `\n`, `\t`, `\r`, `\\`, `\'`, `\"`?
193. What is a raw string and how do you create it?
194. What is the difference between `"Hello\nWorld"` and `r"Hello\nWorld"`?
195. What are Unicode escape sequences?
196. What is the output of `"\u0041"` and `"\U00000041"`?
197. What is the output of `"\x41"`?

5.3 String Indexing and Slicing

198. How do you access individual characters in a string?
199. What is string indexing?
200. Are string indices 0-based or 1-based?
201. Can you use negative indices? What do they represent?
202. What is string slicing?
203. What is the syntax for string slicing?
204. What is the output of `"Hello"[0]`, `"Hello"[-1]`, `"Hello"[1:4]`?
205. What is the output of `"Hello"[::2]` and `"Hello"[::-1]`?
206. What is the difference between `[start:end]` and `[start:end:step]`?
207. What happens if the start or end index is out of range in slicing?
208. What is the output of `"Hello"[10:]` and `"Hello"[10:20]`?

5.4 String Properties and Methods - Part 1

209. What is the `len()` function used for with strings?
210. What is the output of `len("Hello")` and `len("")`?
211. What is the `in` operator used for with strings?
212. What is the output of `"H" in "Hello"`, `"x" in "Hello"`, `"ell" in "Hello"`?
213. What is the `count()` method?
214. What is the output of `"Hello World".count("l")` and `"Hello World".count("o")`?
215. What is the `find()` method?
216. What is the difference between `find()` and `index()`?
217. What is the output of `"Hello World".find("World")` and `"Hello World".find("xyz")`?

218. What is the `startswith()` and `endswith()` method?
219. What is the output of `"Hello".startswith("He")` and `"Hello".endswith("lo")`?

5.5 String Properties and Methods - Part 2

220. What is the `upper()` method?
221. What is the `lower()` method?
222. What is the `capitalize()` method?
223. What is the `title()` method?
224. What is the output of `"Hello World".upper()`, `"Hello World".lower()`, `"hello world".capitalize()`?
225. What is the `strip()` method?
226. What is the difference between `strip()`, `lstrip()`, and `rstrip()`?
227. What is the output of `"  Hello  ".strip()`, `"  Hello  ".lstrip()`, `"  Hello  ".rstrip()`?
228. What is the `replace()` method?
229. What is the output of `"Hello World".replace("World", "Python")`?
230. What is the `split()` method?
231. What is the output of `"Hello World Python".split()` and `"a,b,c".split(",")`?

5.6 String Properties and Methods - Part 3

232. What is the `join()` method?
233. What is the output of `"-".join(["a", "b", "c"])`?
234. What is the difference between `split()` and `join()`?
235. What is the `isdigit()` method?
236. What is the `isalpha()` method?
237. What is the `isalnum()` method?
238. What is the `isspace()` method?
239. What is the `isupper()` and `islower()` method?
240. What is the output of `"123".isdigit()`, `"abc".isalpha()`, `"abc123".isalnum()`?
241. What is the `swapcase()` method?
242. What is the output of `"Hello World".swapcase()`?

5.7 String Formatting

243. What is string formatting?
244. What are the different ways to format strings in Python?

245. What is `%` formatting (old style)?
246. What is the `.format()` method?
247. What are f-strings (formatted string literals)?
248. What is the output of `"%s %d" % ("Hello", 123)`?
249. What is the output of `"{} {}".format("Hello", "World")`?
250. What is the output of `f"Hello {name}"` when `name = "Python"`?
251. What are format specifiers in f-strings?
252. What is the output of `f"{10:05d}"` and `f"{3.14159:.2f}"`?
253. How do you align strings using format specifiers?
254. What is the output of `f"{name:>10}"`, `f"{name:<10}"`, `f"{name:^10}"`?

5.8 String as Sequences

255. Is a string a sequence in Python?
256. Can you iterate over a string?
257. What is the output of `for char in "Hello": print(char)`?
258. Can you modify a string after creation?
259. Why are strings immutable?
260. What happens when you try to change a character in a string?
261. How do you create a modified version of a string?

5.9 Encoding and Decoding

262. What is character encoding?
263. What is UTF-8 encoding?
264. What is ASCII encoding?
265. How do you encode a string to bytes?
266. What is the `encode()` method?
267. What is the `decode()` method?
268. What is the output of `"Hello".encode()` and `"Hello".encode("utf-8")`?
269. What is the difference between `bytes` and `str`?
270. What is Unicode in Python 3?

5.10 Advanced String Concepts

271. What is a regular expression?
272. What is the `re` module?
273. What are some common regex patterns?

274. Can you use `re.search()`, `re.match()`, `re.findall()`?
275. What is the difference between `match()` and `search()`?
276. What is string interpolation?
277. What is the difference between string formatting and string interpolation?
278. How do you handle special characters in strings?

---

6. LIST DATA TYPE

6.1 List Basics

279. What is a list in Python?
280. How do you create a list?
281. What is the syntax for creating a list?
282. Can a list contain different data types?
283. What is the output of `[1, "Hello", 3.14, True]`?
284. Can a list be empty?
285. What is the `list()` function?
286. What is the output of `list("Hello")` and `list(range(5))`?
287. Are lists ordered?
288. Are lists mutable?
289. Can you modify a list after creation?

6.2 List Indexing and Slicing

290. How do you access elements in a list?
291. What is 0-based indexing?
292. Can you use negative indices in a list?
293. What is the output of `[1, 2, 3, 4, 5][0]`, `[1, 2, 3, 4, 5][-1]`?
294. What is list slicing?
295. What is the output of `[1, 2, 3, 4, 5][1:4]` and `[1, 2, 3, 4, 5][::2]`?
296. What is the output of `[1, 2, 3, 4, 5][::-1]`?

6.3 List Methods - Modification

297. What is the `append()` method?
298. What is the output of `lst = [1, 2]; lst.append(3); print(lst)`?
299. What is the `extend()` method?

300. What is the difference between `append()` and `extend()`?
301. What is the output of `lst = [1, 2]; lst.extend([3, 4]); print(lst)`?
302. What is the `insert()` method?
303. What is the output of `lst = [1, 2, 3]; lst.insert(1, 99); print(lst)`?
304. What is the `remove()` method?
305. What is the output of `lst = [1, 2, 3, 2]; lst.remove(2); print(lst)`?
306. What is the `pop()` method?
307. What is the output of `lst = [1, 2, 3]; x = lst.pop(); print(x, lst)`?
308. What is the difference between `remove()` and `pop()`?
309. What is the `clear()` method?
310. What is the output of `lst = [1, 2, 3]; lst.clear(); print(lst)`?

6.4 List Methods - Search and Ordering

311. What is the `index()` method?
312. What is the output of `[1, 2, 3, 2].index(2)`?
313. What is the `count()` method?
314. What is the output of `[1, 2, 3, 2, 2].count(2)`?
315. What is the `sort()` method?
316. What is the output of `lst = [3, 1, 4, 1, 5]; lst.sort(); print(lst)`?
317. What is the `reverse()` method?
318. What is the output of `lst = [1, 2, 3]; lst.reverse(); print(lst)`?
319. What is the difference between `sort()` and `sorted()`?
320. What is the output of `sorted([3, 1, 4, 1, 5])` and `[3, 1, 4, 1, 5]`?

6.5 List Operations

321. What is the `in` operator used for with lists?
322. What is the `len()` function for lists?
323. What is the `+` operator used for with lists?
324. What is the output of `[1, 2] + [3, 4]`?
325. What is the `*` operator used for with lists?
326. What is the output of `[1, 2] * 3`?
327. What is the difference between list concatenation and list repetition?
328. What is the `min()` and `max()` function for lists?
329. What is the `sum()` function for lists?

6.6 List Comprehension

330. What is list comprehension?
331. What is the syntax for list comprehension?
332. What is the output of `[x for x in range(5)]`?
333. What is the output of `[x*2 for x in range(5)]`?
334. How do you use conditions in list comprehension?
335. What is the output of `[x for x in range(10) if x % 2 == 0]`?
336. What is the output of `[x if x % 2 == 0 else -x for x in range(5)]`?
337. What is nested list comprehension?
338. What is the output of `[[y for y in range(3)] for x in range(3)]`?
339. What is the output of `[(x, y) for x in range(3) for y in range(3)]`?
340. How is list comprehension different from using `map()` and `filter()`?

6.7 Nested Lists and 2D Lists

341. What is a nested list?
342. How do you create a 2D list (matrix)?
343. How do you access elements in a nested list?
344. What is the output of `matrix = [[1, 2], [3, 4]]; print(matrix[0][1])`?
345. What is the difference between `[[0]*3]*3` and `[[0 for _ in range(3)] for _ in range(3)]`?
346. Why is `matrix = [[0]*3]*3` problematic?
347. How do you iterate over a 2D list?
348. How do you flatten a nested list?
349. What is the output of `[x for row in [[1, 2], [3, 4]] for x in row]`?

6.8 Copying Lists

350. What is the difference between copying and referencing a list?
351. What is the output of `a = [1, 2]; b = a; b.append(3); print(a)`?
352. How do you create a shallow copy of a list?
353. What is the `copy()` method?
354. What is the output of `a = [1, 2]; b = a.copy(); b.append(3); print(a)`?
355. What is the difference between shallow copy and deep copy?
356. How do you create a deep copy of a nested list?
357. What is the `copy.deepcopy()` function?

7. TUPLE DATA TYPE

7.1 Tuple Basics

358. What is a tuple in Python?
359. How do you create a tuple?
360. What is the syntax for creating a tuple?
361. What is the difference between a tuple and a list?
362. Are tuples mutable or immutable?
363. What is the output of `(1, 2, 3)[0]` and `(1, 2, 3)[-1]`?
364. Can you create a tuple with a single element? How?
365. What is the output of `(1)` vs `(1,)`?
366. Can a tuple be empty?
367. What is the output of `()`?

7.2 Tuple Operations

368. What is tuple indexing?
369. What is tuple slicing?
370. What is the output of `(1, 2, 3, 4, 5)[1:4]` and `(1, 2, 3, 4, 5)[::-1]`?
371. What is the `in` operator used for with tuples?
372. What is the `len()` function for tuples?
373. What is the `+` operator for tuples?
374. What is the `*` operator for tuples?
375. What is the `count()` method for tuples?
376. What is the `index()` method for tuples?
377. Can you modify a tuple after creation? Why or why not?

7.3 Tuple Unpacking

378. What is tuple unpacking?
379. What is the syntax for tuple unpacking?
380. What is the output of `a, b, c = (1, 2, 3); print(a, b, c)`?
381. What is the output of `a, *b, c = (1, 2, 3, 4, 5); print(a, b, c)`?
382. What is the `*` operator in tuple unpacking?
383. What is the output of `*a, = (1, 2, 3); print(a)`?
384. What is the output of `a, b = b, a`? (swapping without temporary variable)

7.4 Tuples as Dictionary Keys and Function Arguments

385. Why can tuples be used as dictionary keys but lists cannot?
386. What is the output of `{(1, 2): "value"}`?
387. What is the output of `{[1, 2]: "value"}`? (Will it work?)
388. Why are immutable objects used as dictionary keys?
389. What is the `*args` in function arguments?
390. What is the output of `def func(*args): print(args)` when called with `func(1, 2, 3)`?

7.5 Named Tuples

391. What is a named tuple?
392. What is the `namedtuple` from the `collections` module?
393. How do you create a named tuple?
394. What is the output of `Point = namedtuple('Point', ['x', 'y']); p = Point(1, 2); print(p.x)`?
395. What is the difference between a regular tuple and a named tuple?
396. What are the advantages of named tuples?

---

8. DICTIONARY DATA TYPE

8.1 Dictionary Basics

397. What is a dictionary in Python?
398. How do you create a dictionary?
399. What is the syntax for creating a dictionary?
400. What are keys and values in a dictionary?
401. What are the requirements for dictionary keys?
402. Can dictionary keys be any data type?
403. What is the output of `{"name": "John", "age": 30}["name"]`?
404. Can dictionary values be any data type?
405. What is the `dict()` function?
406. What is the output of `dict([("a", 1), ("b", 2)])`?
407. Are dictionaries ordered in Python?
408. Since which Python version are dictionaries ordered?
409. Are dictionaries mutable?

410. Can you modify a dictionary after creation?

8.2 Dictionary Access and Modification

411. How do you access a value in a dictionary?
412. What is the output of `dict = {"a": 1, "b": 2}; print(dict["a"])`?
413. What happens if you access a key that doesn't exist?
414. How do you add a new key-value pair?
415. What is the output of `dict = {}; dict["new_key"] = "new_value"; print(dict)`?
416. How do you modify a value for an existing key?
417. How do you delete a key-value pair?
418. What is the `del` statement?
419. What is the `pop()` method for dictionaries?
420. What is the difference between `del` and `pop()`?
421. What is the `clear()` method for dictionaries?
422. What is the output of `dict = {"a": 1}; dict.clear(); print(dict)`?

8.3 Dictionary Methods - Access

423. What is the `get()` method?
424. What is the output of `dict = {"a": 1}; print(dict.get("a"))` and `print(dict.get("b"))`?
425. What is the difference between `dict["key"]` and `dict.get("key")`?
426. What is the `keys()` method?
427. What is the output of `dict = {"a": 1, "b": 2}; print(dict.keys())`?
428. What is the `values()` method?
429. What is the output of `dict = {"a": 1, "b": 2}; print(dict.values())`?
430. What is the `items()` method?
431. What is the output of `dict = {"a": 1, "b": 2}; print(dict.items())`?
432. What is the `in` operator for dictionaries?
433. What is the output of `"a" in {"a": 1, "b": 2}`?
434. What is the `update()` method?
435. What is the output of `dict1 = {"a": 1}; dict1.update({"b": 2}); print(dict1)`?

8.4 Dictionary Iteration

436. How do you iterate over dictionary keys?
437. What is the output of `for key in {"a": 1, "b": 2}: print(key)`?

438. How do you iterate over dictionary values?
439. What is the output of `for value in {"a": 1, "b": 2}.values(): print(value)`?
440. How do you iterate over key-value pairs?
441. What is the output of `for k, v in {"a": 1, "b": 2}.items(): print(k, v)`?
442. What is the difference between iterating over `dict`, `dict.keys()`, and `dict.items()`?

8.5 Dictionary Comprehension

443. What is dictionary comprehension?
444. What is the syntax for dictionary comprehension?
445. What is the output of `{x: x**2 for x in range(5)}`?
446. What is the output of `{k: v for k, v in [("a", 1), ("b", 2)]}`?
447. How do you use conditions in dictionary comprehension?
448. What is the output of `{x: x**2 for x in range(10) if x % 2 == 0}`?

8.6 Nested Dictionaries

449. What is a nested dictionary?
450. How do you create a nested dictionary?
451. How do you access values in a nested dictionary?
452. What is the output of `dict = {"a": {"b": 1}}; print(dict["a"]["b"])`?
453. How do you modify values in a nested dictionary?
454. How do you iterate over a nested dictionary?

8.7 Dictionary Copying

455. What is the difference between copying and referencing a dictionary?
456. How do you create a shallow copy of a dictionary?
457. What is the `copy()` method for dictionaries?
458. What is the difference between shallow copy and deep copy for dictionaries?
459. How do you create a deep copy of a nested dictionary?

8.8 Advanced Dictionary Concepts

460. What is the `setdefault()` method?
461. What is the output of `dict = {}; dict.setdefault("a", 1); print(dict)`?
462. What is the `fromkeys()` method?
463. What is the output of `dict.fromkeys(["a", "b", "c"], 0)`?
464. What is the `defaultdict` from the `collections` module?

465. What is the `OrderedDict` from the `collections` module?
466. What is `Counter` from the `collections` module?
467. What is `ChainMap` from the `collections` module?

---

9. SET DATA TYPE

9.1 Set Basics

468. What is a set in Python?
469. How do you create a set?
470. What is the syntax for creating a set?
471. What is the difference between a set and a list?
472. What is the difference between a set and a dictionary?
473. Are sets mutable?
474. Are sets ordered?
475. Can sets contain duplicate elements?
476. What is the output of `{1, 2, 2, 3}` and `len({1, 2, 2, 3})`?
477. Can a set be empty? How do you create an empty set?
478. What is the output of `{}` and `set()`?
479. What is the `set()` function?
480. What is the output of `set("Hello")` and `set([1, 2, 2, 3])`?

9.2 Set Operations

481. What is the `in` operator for sets?
482. What is the `len()` function for sets?
483. What is the `add()` method?
484. What is the output of `s = {1, 2}; s.add(3); print(s)`?
485. What is the `remove()` method?
486. What is the `discard()` method?
487. What is the difference between `remove()` and `discard()`?
488. What is the `pop()` method for sets?
489. What is the `clear()` method for sets?
490. What is the `copy()` method for sets?

9.3 Set Mathematical Operations

491. What is set union?
492. What is the `union()` method and `|` operator?
493. What is the output of `{1, 2} | {2, 3}` and `{1, 2}.union({2, 3})`?
494. What is set intersection?
495. What is the `intersection()` method and `&` operator?
496. What is the output of `{1, 2} & {2, 3}` and `{1, 2}.intersection({2, 3})`?
497. What is set difference?
498. What is the `difference()` method and `-` operator?
499. What is the output of `{1, 2} - {2, 3}` and `{1, 2}.difference({2, 3})`?
500. What is symmetric difference?
501. What is the `symmetric_difference()` method and `^` operator?
502. What is the output of `{1, 2} ^ {2, 3}`?

9.4 Set Comparisons

503. What is the `issubset()` method?
504. What is the `issuperset()` method?
505. What is the `isdisjoint()` method?
506. What is the output of `{1, 2}.issubset({1, 2, 3})`?
507. What is the output of `{1, 2, 3}.issuperset({1, 2})`?
508. What is the output of `{1, 2}.isdisjoint({3, 4})`?
509. What is the `<`, `<=`, `>`, `>=` operators for sets?
510. What is the output of `{1, 2} < {1, 2, 3}`?

9.5 Set Comprehension

511. What is set comprehension?
512. What is the syntax for set comprehension?
513. What is the output of `{x for x in range(5)}`?
514. What is the output of `{x**2 for x in range(5)}`?
515. How do you use conditions in set comprehension?
516. What is the output of `{x for x in range(10) if x % 2 == 0}`?

9.6 Frozen Sets

517. What is a frozen set?
518. How do you create a frozen set?

519. What is the `frozenset()` function?
520. What is the output of `frozenset([1, 2, 2, 3])`?
521. What is the difference between a set and a frozen set?
522. Why would you use a frozen set?
523. Can frozen sets be used as dictionary keys?
524. What operations are available for frozen sets?

---

10. BYTES AND BYTEARRAY DATA TYPES

10.1 Bytes Basics

525. What are bytes in Python?
526. How do you create a bytes object?
527. What is the `b` prefix used for?
528. What is the output of `b"Hello"` and `type(b"Hello")`?
529. Are bytes mutable or immutable?
530. What is the `bytes()` function?
531. What is the output of `bytes([72, 101, 108, 108, 111])`?
532. What is the output of `bytes("Hello", "utf-8")`?
533. What is the difference between `str` and `bytes`?
534. What is the output of `"Hello".encode()` and `"Hello".encode("utf-8")`?
535. What is the output of `b"Hello".decode()`?

10.2 Bytes Operations

536. Can you access individual bytes like you do with lists?
537. What is the output of `b"Hello"[0]`?
538. Can you slice bytes?
539. What is the output of `b"Hello"[1:4]`?
540. What is the `in` operator for bytes?
541. What is the `len()` function for bytes?
542. Can you concatenate bytes?
543. What is the output of `b"Hello" + b" World"`?
544. Can you repeat bytes?
545. What is the output of `b"Hi" * 3`?

10.3 Bytearray Basics

546. What is a bytearray?
547. How do you create a bytearray?
548. What is the `bytearray()` function?
549. What is the output of `bytearray(b"Hello")`?
550. What is the difference between `bytes` and `bytearray`?
551. Are bytearrays mutable?
552. Can you modify a bytearray after creation?
553. What is the output of `b = bytearray(b"Hello"); b[0] = 74; print(b)`?

10.4 Bytearray Operations

554. What methods are available for bytearrays?
555. What is the `append()` method for bytearrays?
556. What is the `extend()` method for bytearrays?
557. What is the `remove()` method for bytearrays?
558. What is the `pop()` method for bytearrays?
559. What is the `insert()` method for bytearrays?
560. What is the `clear()` method for bytearrays?

---

11. NONE DATA TYPE

11.1 None Basics

561. What is `None` in Python?
562. What is the purpose of `None`?
563. How do you assign `None` to a variable?
564. What is the type of `None`?
565. What is the output of `type(None)`?
566. What is the output of `None == None` and `None is None`?
567. Is `None` mutable or immutable?
568. What is the output of `bool(None)`?
569. What is the output of `if None: print("True") else: print("False")`?

11.2 Using None

570. When do you use `None` in programming?
571. How do you check if a variable is `None`?
572. What is the output of `x = None; if x is None: print("None")`?
573. What is the difference between `== None` and `is None`?
574. Which should you use: `== None` or `is None`? Why?
575. What is the output of `x = None; print(x or "Default")`?
576. Can functions return `None`?
577. What is the output of a function with no return statement?

---

12. RANGE DATA TYPE

12.1 Range Basics

578. What is a range in Python?
579. What is the `range()` function?
580. What is the syntax for `range()`?
581. What is the output of `range(5)`?
582. What is the output of `range(1, 5)`?
583. What is the output of `range(0, 10, 2)`?
584. What is the type of `range(5)`?
585. Are ranges mutable or immutable?
586. Can you index a range?
587. What is the output of `range(10)[5]`?

12.2 Range Operations

588. Can you iterate over a range?
589. What is the output of `for i in range(5): print(i)`?
590. Can you convert a range to a list?
591. What is the output of `list(range(5))` and `list(range(1, 6))`?
592. What is the `in` operator for ranges?
593. What is the output of `5 in range(10)`?
594. What is the `len()` function for ranges?
595. What is the output of `len(range(10))`?
596. Can you slice a range?

PART 3: TYPE CONVERSION AND COMPARISON

13. Type Conversion (Casting)

13.1 Explicit Type Conversion

597. What is type conversion (casting)?
598. What are the main type conversion functions in Python?
599. What is the `int()` function and what does it return?
600. What is the output of `int(3.7)`, `int("123")`, `int(True)`?
601. What is the `float()` function?
602. What is the output of `float(5)`, `float("3.14")`, `float(True)`?
603. What is the `str()` function?
604. What is the output of `str(123)`, `str(3.14)`, `str(True)`?
605. What is the `bool()` function?
606. What is the output of `bool(0)`, `bool(1)`, `bool("")`, `bool("Hello")`?
607. What is the `list()` function?
608. What is the output of `list("Hello")` and `list((1, 2, 3))`?
609. What is the `tuple()` function?
610. What is the output of `tuple([1, 2, 3])` and `tuple("Hello")`?
611. What is the `set()` function?
612. What is the output of `set([1, 2, 2, 3])` and `set("Hello")`?
613. What is the `dict()` function?

13.2 Implicit Type Conversion

614. What is implicit type conversion?
615. When does Python perform implicit type conversion?
616. What is the output of `5 + 3.14`?
617. What is the output of `"Hello" + " " + "World"`?
618. What is the output of `"5" + 3`? Does it work?
619. What is the output of `True + 5`?
620. What is the output of `[1, 2] + [3, 4]`? Does it work?
621. What is the output of `[1, 2] + "Hello"`? Does it work?

13.3 Errors in Type Conversion

622. What happens when you try invalid type conversions?

623. What is the output of `int("123.45")`? Does it work?
624. What is the output of `int("abc")`? Does it work?
625. What error is raised for invalid conversions?
626. How do you handle type conversion errors?

---

14. Type Checking and Comparison

14.1 Type Checking

627. How do you check the type of a variable?
628. What is the `type()` function?
629. What is the output of `type(5)`, `type(3.14)`, `type("Hello")`, `type([1, 2])`?
630. What is the `isinstance()` function?
631. What is the output of `isinstance(5, int)` and `isinstance("Hello", str)`?
632. What is the difference between `type()` and `isinstance()`?
633. When should you use `isinstance()` over `type()`?
634. Can you check multiple types with `isinstance()`?
635. What is the output of `isinstance(5, (int, float))`?

14.2 Equality vs Identity

636. What is the difference between `==` and `is`?
637. What does `==` check?
638. What does `is` check?
639. What is the output of `5 == 5` and `5 is 5`?
640. What is the output of `"Hello" == "Hello"` and `"Hello" is "Hello"`?
641. What is the output of `[1, 2] == [1, 2]` and `[1, 2] is [1, 2]`?
642. What is the output of `a = [1, 2]; b = a; a is b`?
643. When should you use `==` and when should you use `is`?
644. Why is `x is None` preferred over `x == None`?

PART 4: DATA TYPE CHARACTERISTICS AND SELECTION

15. Mutable vs Immutable Data Types

15.1 Understanding Mutability

645. What is a mutable data type?
646. What is an immutable data type?
647. List the immutable data types in Python.
648. List the mutable data types in Python.
649. What is the difference between modifying a variable and reassigning it?
650. What is the output of `x = 5; x = 10; print(x)`?
651. What is the output of `x = [1, 2]; x.append(3); print(x)`?

15.2 Implications of Mutability

652. Why does it matter if a data type is mutable?
653. What is the output of `a = [1, 2]; b = a; b.append(3); print(a)`?
654. Why doesn't this happen with immutable types?
655. What is the output of `a = 5; b = a; b = 10; print(a)`?
656. What are the implications for function arguments?
657. What is the output of `def func(lst): lst.append(4); my_list = [1, 2, 3]; func(my_list); print(my_list)`?
658. Can you use mutable objects as dictionary keys? Why or why not?
659. Can you use immutable objects as dictionary keys?
660. What is the output of `{[1, 2]: "value"}`?
661. What is the output of `{(1, 2): "value"}`?

---

16. Choosing the Right Data Type

16.1 When to Use Each Data Type

662. When should you use a list instead of a tuple?
663. When should you use a tuple instead of a list?
664. When should you use a set instead of a list?
665. When should you use a dictionary instead of a list?
666. What are the performance implications of each data type?
667. How does the choice of data type affect memory usage?
668. Which data type has the fastest lookup time?
669. Which data type has the fastest insertion time?
670. Which data type has the fastest deletion time?

16.2 Data Type Performance

671. What is the time complexity of list operations (access, insertion, deletion)?
672. What is the time complexity of tuple operations?
673. What is the time complexity of set operations?
674. What is the time complexity of dictionary operations?
675. How does Python implement dictionaries and sets?
676. What is hashing in Python?
677. Why are sets and dictionaries faster for membership testing?
678. What is the Big O notation?

---

17. Complex Data Structures

17.1 Combinations and Nested Structures

679. Can you have a list of dictionaries?
680. What is the output of `[{"name": "John", "age": 30}, {"name": "Jane", "age": 25}]`?
681. Can you have a dictionary of lists?
682. What is the output of `{"names": ["John", "Jane"], "ages": [30, 25]}`?
683. Can you have a set of tuples?
684. What is the output of `{(1, 2), (3, 4), (1, 2)}`?
685. What is the output of `{(1, 2), (3, [4, 5])}`? (Can you have a list inside a tuple in a set?)
686. What are the advantages and disadvantages of nested structures?
687. How do you access deeply nested values?
688. What is the output of `data = [{"items": [1, 2, 3]}]; print(data[0]["items"][1])`?

17.2 Data Structure Serialization

689. What is serialization?
690. What is the `json` module used for?
691. What is the `pickle` module used for?
692. What is the difference between `json` and `pickle`?
693. How do you convert a Python object to JSON?
694. What is the `json.dumps()` function?
695. What is the `json.dump()` function?
696. What is the `json.loads()` function?

697. What is the `json.load()` function?

PART 5: ADVANCED DATA TYPE CONCEPTS

18. Special Methods and Dunder Methods

18.1 Magic Methods for Data Types

698. What are magic methods (dunder methods)?
699. What is the `__init__()` method?
700. What is the `__str__()` method?
701. What is the `__repr__()` method?
702. What is the difference between `__str__()` and `__repr__()`?
703. What is the `__len__()` method?
704. What is the `__getitem__()` method?
705. What is the `__setitem__()` method?
706. What is the `__delitem__()` method?
707. What is the `__iter__()` method?
708. What is the `__next__()` method?
709. What is the `__contains__()` method?
710. What is the `__eq__()`, `__lt__()`, `__le__()`, `__gt__()`, `__ge__()` methods?

18.2 Custom Data Type Creation

711. How do you create a custom data type (class)?
712. How do you define `__init__()` for initialization?
713. How do you define methods for your custom type?
714. How do you make your custom type printable?
715. How do you make your custom type iterable?
716. How do you make your custom type support indexing?
717. How do you make your custom type support comparison?
718. What are properties and how do you create them?

---

19. Advanced Iteration and Generators

19.1 Iterables and Iterators

719. What is an iterable?
720. What is an iterator?
721. What is the difference between an iterable and an iterator?
722. What is the `iter()` function?
723. What is the `next()` function?
724. What is the output of `it = iter([1, 2, 3]); next(it); next(it); next(it); next(it)`?
725. What happens when an iterator is exhausted?
726. What is the `StopIteration` exception?

19.2 Generators

727. What is a generator?
728. How do you create a generator function?
729. What is the `yield` keyword?
730. What is the difference between a generator and a regular function?
731. What are the advantages of generators?
732. How do you use generators in for loops?
733. What is generator expression?
734. What is the syntax for generator expression?
735. What is the output of `(x**2 for x in range(5))`?
736. What is the difference between list comprehension and generator expression?

19.3 More Complex Iteration

737. What is the `zip()` function?
738. What is the output of `zip([1, 2, 3], ['a', 'b', 'c'])`?
739. What is the `enumerate()` function?
740. What is the output of `enumerate(['a', 'b', 'c'])`?
741. What is the `map()` function?
742. What is the output of `map(lambda x: x**2, [1, 2, 3])`?
743. What is the `filter()` function?
744. What is the output of `filter(lambda x: x > 2, [1, 2, 3, 4, 5])`?
745. What is the `reduce()` function?

---

20. Useful Data Type Patterns and Tricks

20.1 Common Data Type Patterns

746. What is the pattern for removing duplicates from a list?
747. What is the pattern for counting occurrences in a list?
748. What is the pattern for finding the most common element?
749. What is the pattern for flattening nested lists?
750. What is the pattern for transposing a matrix (2D list)?
751. What is the pattern for creating a frequency dictionary?
752. What is the pattern for grouping items by a key?
753. What is the pattern for sorting dictionaries?
754. What is the pattern for sorting lists of tuples or objects?
755. What is the pattern for merging multiple dictionaries?

20.2 Memory and Performance Optimization

756. How can you optimize memory usage with data types?
757. When should you use generators instead of lists?
758. What is the memory difference between a list and a generator?
759. How does `__slots__` reduce memory usage in classes?
760. What are data structure trade-offs?
761. When should you use a set instead of a list for membership testing?
762. What is the performance impact of deep copying vs shallow copying?
763. How can you optimize dictionary lookup time?
764. What is the difference between pre-allocated lists and dynamic growth?

PART 6: PRACTICAL SCENARIOS AND EXERCISES

21. Real-World Use Cases

21.1 Data Processing Scenarios

765. How would you process a list of dictionaries representing students with their grades?
766. How would you find the average score of students?
767. How would you filter students who scored above 80?
768. How would you group students by their class?
769. How would you create a list of tuples (name, grade) and sort by grade?
770. How would you remove duplicates from a large list?

771. How would you count word frequencies in a text?
772. How would you create a frequency table using a dictionary?
773. How would you find the most common element in a list?
774. How would you merge two lists without duplicates?

21.2 Data Validation and Cleaning

775. How would you validate if all items in a list are of the same type?
776. How would you remove None values from a list?
777. How would you clean whitespace from string data?
778. How would you validate an email address format?
779. How would you check if a string is alphanumeric?
780. How would you convert a list of strings to numbers, handling errors?
781. How would you normalize data (e.g., convert to lowercase)?
782. How would you remove outliers from numerical data?
783. How would you handle missing data in a dictionary?

21.3 Data Transformation

784. How would you transpose a matrix?
785. How would you flatten a nested list?
786. How would you convert between different data structures?
787. How would you pivot data from one structure to another?
788. How would you convert a dictionary to a formatted string?
789. How would you convert a CSV line to a dictionary?
790. How would you split a string into a list of words?
791. How would you join list items into a single string?
792. How would you create a lookup table from two lists?

---

22. Interview Coding Challenges

22.1 String Challenges

793. Write code to check if a string is a palindrome.
794. Write code to check if two strings are anagrams.
795. Write code to find the longest word in a sentence.
796. Write code to count vowels in a string.
797. Write code to reverse a string without using `[::-1]`.

798. Write code to find the first non-repeating character in a string.
799. Write code to implement simple string compression (e.g., "aaa" → "a3").
800. Write code to check if a string contains all unique characters.

22.2 List Challenges

801. Write code to find the second largest element in a list.
802. Write code to find the intersection of two lists.
803. Write code to find the union of two lists.
804. Write code to remove duplicates while preserving order.
805. Write code to find all pairs that sum to a target value.
806. Write code to rotate a list by k positions.
807. Write code to find the maximum sum of a contiguous subarray.
808. Write code to check if a list is a rotation of another list.

22.3 Dictionary Challenges

809. Write code to count word frequencies from a list of words.
810. Write code to group items by a property.
811. Write code to find the intersection of two dictionaries.
812. Write code to merge dictionaries and handle duplicate keys.
813. Write code to invert a dictionary (swap keys and values).
814. Write code to find keys with the maximum value.
815. Write code to create a dictionary from two lists (keys and values).
816. Write code to filter a dictionary by value.

22.4 Mixed Data Type Challenges

817. Write code to find duplicates in a list of tuples.
818. Write code to convert a list of tuples to a dictionary.
819. Write code to find common elements in multiple lists.
820. Write code to create a nested dictionary from a flat list.
821. Write code to flatten a nested list of varying depths.
822. Write code to sort a list of dictionaries by multiple keys.
823. Write code to group records by multiple fields.
824. Write code to transform a list of records into a pivot table.

23.1 Predict the Output

825. What is the output?

```python
x = [1, 2, 3]
y = x
y.append(4)
print(x)
```

826. What is the output?

```python
x = (1, 2, [3, 4])
x[2].append(5)
print(x)
```

827. What is the output?

```python
lst = [[0] * 3] * 3
lst[0][0] = 1
print(lst)
```

828. What is the output?

```python
d = {"a": [1, 2], "b": [3, 4]}
d["a"].append(5)
print(d)
```

829. What is the output?

```python
s = {1, 2, 3}
s.add(2)
print(len(s))
```

830. What is the output?

```python
x = "hello"
x[0] = "H"
print(x)
```

23.2 Debugging Scenarios

831. What's wrong with this code? How would you fix it?

```python
my_list = [1, 2, 3]
my_list[10] = 5
```

832. What's wrong with this code? How would you fix it?

```python
try:
    x = int("abc")
except:
    print("Error")
```

833. What's wrong with this code? How would you fix it?

```python
d = {[1, 2]: "value"}
```

834. What's wrong with this code? How would you fix it?

```python
my_dict = {}
my_dict.append(1)
```

835. What's wrong with this code? How would you fix it?

```python
```

```
s = {1, 2, 3}
x = s[0]
```

SUMMARY AND QUICK REFERENCE

Data Type Comparison Table

Type	Mutable	Ordered	Indexed	Duplicates	Use Case
int	No	-	-	-	Whole numbers
float	No	-	-	-	Decimal numbers
str	No	Yes	Yes	Yes	Text
list	Yes	Yes	Yes	Yes	Ordered collection
tuple	No	Yes	Yes	Yes	Immutable collection
set	Yes	No	No	No	Unique items
dict	Yes	Yes (3.7+)	By key	No (for keys)	Key-value pairs
bool	No	-	-	-	True/False
None	No	-	-	-	Null value
bytes	No	Yes	Yes	Yes	Binary data
bytearray	Yes	Yes	Yes	Yes	Mutable binary

Key Concepts to Remember

1. **Mutability** - Lists, sets, dicts are mutable; strings, tuples, ints are immutable
2. **Pass by Reference** - Mutable objects are passed by reference to functions
3. **Type Checking** - Use `isinstance()` instead of `type()` for flexibility
4. **Identity vs Equality** - Use `is` for `None` and `is not` for singletons
5. **String Immutability** - Strings cannot be modified; you must create new strings
6. **List vs Tuple** - Use tuples for immutable, hashable collections
7. **Set Operations** - Sets excel at membership testing and removing duplicates
8. **Dictionary Keys** - Keys must be immutable and hashable
9. **Shallow vs Deep Copy** - Be careful with nested structures

10. **Time Complexity** - Choose data structures based on your access patterns

---

Total Questions: 835 Difficulty Range: Beginner to Advanced Last Updated: 2026